



GL- k curves: a family of polynomial curves for intuitive modelling

Andriamahenina Ramanantoanina^{a, b}, Kai Hormann^{b, *}

^a *Kajiy University, Antananarivo, Madagascar*

^b *Università della Svizzera italiana, Lugano, Switzerland*

ARTICLE INFO

Keywords:

Bézier curves

Legendre polynomials

Line art

ABSTRACT

Bézier curves are typically used for interactively designing polynomial curves, but it is well known that this becomes increasingly unintuitive as the degree of the curve grows. Gauss–Legendre (GL) curves have been proposed as an alternative for modelling high-degree polynomial curves. They come with the advantage of preserving a close relationship between the curve and its control polygon, but are costly to evaluate in their native form. We first show how to express GL curves in the Legendre basis. Building on this and taking advantage of the three-term recurrence relation of Legendre polynomials, we explore a linear-time algorithm for evaluating GL curves. We further introduce a more general family of GL- k curves, with GL curves corresponding to the case $k = 0$ and observe that GL-1 curves are often remarkably similar to cubic B-spline curves. Unlike Bézier curves, the start and the end of a GL- k curve are not tangent to the first and the last edge of the control polygon, respectively, and we present a strategy for restoring this property.

1. Introduction

For the purpose of curve design, a polynomial curve p is commonly expressed in Bézier form. However, as illustrated in Fig. 2 (left), modelling high-degree curves using Bézier control polygons is often not very intuitive, because the relation between curve and control polygon weakens as the degree increases. Two general approaches have been developed to address this challenge. On the one hand, we can give up the concept of a single polynomial curve and work with piecewise polynomial curves instead, leading to the well-established use of splines (e.g., B-splines (Prautzsch et al., 2002), κ -curves (Yan et al., 2017) and their G^2 alternatives (Jiang and Chen, 2025)). On the other hand, we can give up one or more of the properties of the Bernstein basis (e.g., the non-negativity that gives rise to the *convex hull property* of Bézier curves) and adopt alternative polynomial bases. In this paper, we focus on the second option (Fig. 1).

One such alternative basis are the Lagrange polynomials, which allow the curve to be expressed in terms of control points that are interpolated by p (see Fig. 2, middle). However, high-degree Lagrange polynomials can suffer from significant oscillations near the endpoints, especially when uniformly spaced nodes are used, a behaviour known as Runge's phenomenon (Runge, 1901). As a result, even small adjustments of the control points can create large and undesirable artefacts near the endpoints. One can attempt to reduce these oscillations using distance-based node distributions such as centripetal (Lee, 1989) or chordal parametrizations (Hartley and Judd, 1978), but producing visually pleasing curves still requires a very skilled and experienced designer.

Another option is provided by the recently introduced Gauss–Legendre curves (Kim and Moon, 2017; Moon et al., 2020, 2023), which are defined by control polygons that are geometrically close to the curve, even for high degree (see Fig. 2, right). A planar Gauss–Legendre (GL) curve is a polynomial curve $p: [-1, 1] \rightarrow \mathbb{R}^2$ of degree n , defined by $n+1$ control points $p_0, \dots, p_n \in \mathbb{R}^2$, such that the tangent vectors $p'(\tau_i)$ at the roots $\tau_0, \dots, \tau_{n-1}$ of the degree- n Legendre polynomial P_n are parallel to the edge vectors $v_i = p_{i+1} - p_i$.

* Corresponding author.

Email address: kai.hormann@usi.ch (K. Hormann).

<https://doi.org/10.1016/j.cagd.2026.102558>

Received 9 March 2026; Accepted 9 April 2026

Available online 14 April 2026

0167-8396/© 2026 The Author(s). Published by Elsevier B.V. This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>).

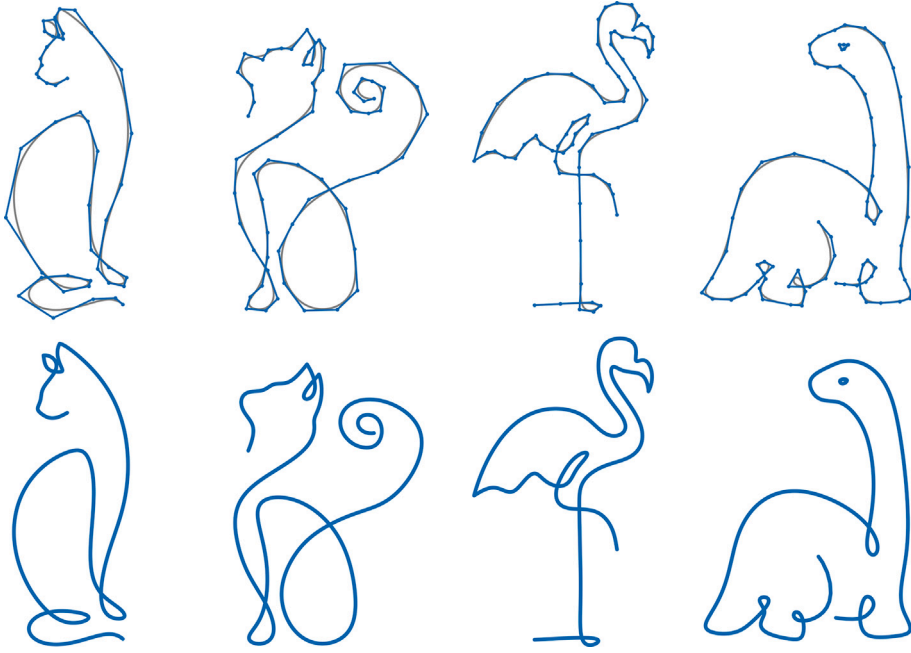


Fig. 1. Examples of line art (bottom) and the corresponding control polygons (top), designed using GL-1 curves.

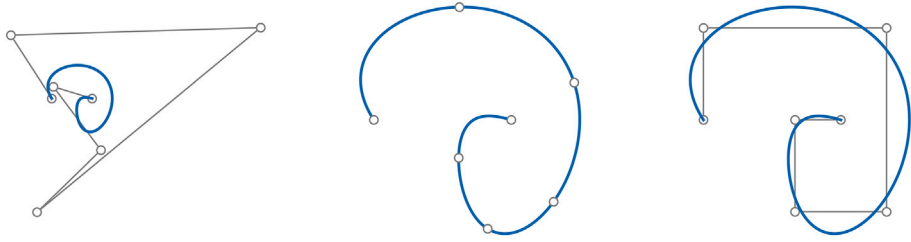


Fig. 2. Example of the same polynomial curve of degree 6, represented as a Bézier curve (left), an interpolatory curve (middle), and a GL curve (right).

A GL curve can be expressed as

$$p(t) = \sum_{i=0}^n p_i F_i^n(t), \quad F_i^n(t) = G_{i-1}^n(t) - G_i^n(t), \quad i = 0, \dots, n,$$

where

$$G_{-1}^n(t) = \frac{1}{2}, \quad G_i^n(t) = \frac{\int_{-1}^t \ell_i(s) ds}{\int_{-1}^1 \ell_i(s) ds} - \frac{1}{2}, \quad i = 0, \dots, n-1, \quad G_n^n(t) = -\frac{1}{2} \quad (1)$$

and the functions ℓ_i are the Lagrange polynomials

$$\ell_i(t) = \prod_{j=0, j \neq i}^{n-1} \frac{t - \tau_j}{\tau_i - \tau_j}, \quad i = 0, \dots, n-1. \quad (2)$$

A natural property that is desired when designing with control polygons is that the curve p starts at p_0 and ends at p_n . This *endpoint interpolation property* is satisfied by GL curves. Indeed, since $G_i^n(-1) = -\frac{1}{2}$ for $i = 0, \dots, n$, we find that $F_0^n(-1) = 1$ and $F_i(-1) = 0$ for $i = 1, \dots, n$, hence $p(-1) = p_0$. Similarly, since $G_i^n(1) = \frac{1}{2}$ for $i = -1, \dots, n-1$, we deduce that $p(1) = p_n$. Moreover, one can verify that

$$p'(\tau_i) = \frac{v_i}{\int_{-1}^1 \ell_i(s) ds}, \quad i = 0, \dots, n-1. \quad (3)$$

Note that unlike Bézier curves, the tangent vectors of p at $t = -1$ and $t = 1$ are not parallel to v_0 and v_{n-1} , that is, GL curves do not have the *endpoint tangent property*. This property can be obtained by using an alternative quadrature-based polynomial basis (Kim and Moon, 2019).

1.1. Contributions

The evaluation of a GL curve involves the computation of definite integrals in (1), and a naive algorithm requires $O(n^3)$ operations. In Section 2, we show how to express GL curves in the Legendre basis. We then take advantage of this expression to develop a linear-time evaluation algorithm in Section 3. In Section 4, inspired by (Ramanantoanina and Hormann, 2024), we extend the construction of GL curves to derive a whole family of GL- k curves (for $k \in \mathbb{N}_0$), and we prove some properties of these curves in Section 5. Finally, in Section 6, we develop a technique that allows for manipulating the curve's tangents at the endpoints and restores the endpoint tangent property of GL- k curves.

2. GL curves in the Legendre basis

While the polynomials G_i^n in (1) can be evaluated by means of quadrature rules, this becomes computationally expensive as the degree n grows. We propose instead to express GL curves in the Legendre basis. This is motivated by the fact that, contrary to Lagrange polynomials, simple formulas for the integrals of the Legendre polynomials P_i are known (Arfken et al., 2012, Eq. (12.23)), namely

$$\int_{-1}^t P_i(s) ds = \frac{P_{i+1}(s) - P_{i-1}(s)}{2i+1} \Big|_{-1}^t = \frac{P_{i+1}(t) - P_{i-1}(t)}{2i+1} + \delta_{0,i}, \quad i \geq 0, \quad (4)$$

where $\delta_{i,j}$ is the Kronecker delta and following the convention that $P_{-1}(t) = 0$. Hence, it suffices to express the Lagrange polynomials ℓ_i in (2) in terms of Legendre polynomials and to then derive the coefficients of p in that basis.

Proposition 1. *Let $i, n \in \mathbb{N}_0$ with $0 \leq i < n$ and consider the Lagrange polynomial ℓ_i in (2). Then,*

$$\ell_i(t) = \sum_{j=0}^{n-1} c_j P_j(t), \quad c_j = \frac{(2j+1)P_j(\tau_i)}{\sum_{k=0}^{n-1} (2k+1)P_k(\tau_i)^2}, \quad j = 0, \dots, n-1,$$

where $\tau_0, \dots, \tau_{n-1}$ are the roots of P_n .

Proof. First, recall (Henrici, 1964) that we can express $\ell_i(t)$ as

$$\ell_i(t) = \frac{P_n(t)}{(t - \tau_i)P_n'(\tau_i)}, \quad i = 0, \dots, n-1. \quad (5)$$

Next, consider the Christoffel–Darboux kernel $K_{n-1}(t, s)$ (Simon, 2008, Eq. (1.12)), defined in terms of the Legendre polynomials,

$$K_{n-1}(t, s) = \frac{1}{2} \sum_{j=0}^{n-1} (2j+1)P_j(t)P_j(s). \quad (6)$$

The Christoffel–Darboux formula (Abramowitz and Stegun, 1965, Eq. (22.12.1)) then states that

$$K_{n-1}(t, s) = \frac{2(n-1)+1}{2} \cdot \frac{a_{n-1}}{a_n} \cdot \frac{P_{n-1}(s)P_n(t) - P_n(s)P_{n-1}(t)}{t-s},$$

where $a_k = \frac{1}{2^k} \binom{2k}{k}$ is the leading coefficient of P_k . Since τ_i is a root of P_n , we have

$$K_{n-1}(t, \tau_i) = \lambda_n \frac{P_n(t)}{t - \tau_i}, \quad K_{n-1}(\tau_i, \tau_i) = \lambda_n P_n'(\tau_i), \quad \lambda_n = \frac{2(n-1)+1}{2} \frac{a_{n-1}}{a_n} P_{n-1}(\tau_i), \quad (7)$$

and we finally deduce from (5) and (7) that $\ell_i(t)$ is given by the ratio

$$\ell_i(t) = \frac{K_{n-1}(t, \tau_i)}{K_{n-1}(\tau_i, \tau_i)}. \quad (8)$$

The statement then follows from (6). □

Using Proposition 1 and Eq. (1) and (4), we can now express G_i^n in the Legendre basis.

Proposition 2. *Let $i, n \in \mathbb{N}_0$ with $0 \leq i < n$. Then, $G_i^n(t)$ in (1) can be written as*

$$G_i^n(t) = \frac{1}{2} \left(\sum_{j=0}^{n-2} (P_{j-1}(\tau_i) - P_{j+1}(\tau_i)) P_j(t) + \sum_{j=n-1}^n P_{j-1}(\tau_i) P_j(t) \right).$$

Proof. We first notice from (1) and (8) that

$$G_i^n(t) + \frac{1}{2} = \frac{\int_{-1}^t \ell_i(s) ds}{\int_{-1}^1 \ell_i(s) ds} = \frac{\int_{-1}^t K_{n-1}(s, \tau_i) ds}{\int_{-1}^1 K_{n-1}(s, \tau_i) ds}.$$

It further follows from (4) and (6) that

$$\begin{aligned}
 2 \int_{-1}^t K_{n-1}(s, \tau_i) ds &= \sum_{j=0}^{n-1} (2j+1) P_j(\tau_i) \int_{-1}^t P_j(s) ds \\
 &= 1 + \sum_{j=0}^{n-1} P_j(\tau_i) (P_{j+1}(t) - P_{j-1}(t)) \\
 &= 1 + \sum_{j=1}^n P_{j-1}(\tau_i) P_j(t) - \sum_{j=-1}^{n-2} P_{j+1}(\tau_i) P_j(t) \\
 &= 1 - \tau_i + \sum_{j=1}^{n-2} (P_{j-1}(\tau_i) - P_{j+1}(\tau_i)) P_j(t) + \sum_{j=n-1}^n P_{j-1}(\tau_i) P_j(t)
 \end{aligned}$$

and similarly

$$\begin{aligned}
 2 \int_{-1}^1 K_{n-1}(s, \tau_i) ds &= 1 - \tau_i + \sum_{j=1}^{n-2} (P_{j-1}(\tau_i) - P_{j+1}(\tau_i)) P_j(1) + \sum_{j=n-1}^n P_{j-1}(\tau_i) P_j(1) \\
 &= 2 - \sum_{j=0}^1 P_{j+1}(\tau_i) + \sum_{j=2}^{n-2} (P_{j-1}(\tau_i) - P_{j+1}(\tau_i)) + \sum_{j=n-1}^n P_{j-1}(\tau_i) \\
 &= 2.
 \end{aligned}$$

Hence, we deduce that

$$G_i^n(t) + \frac{1}{2} = \frac{1}{2} \left(1 + \sum_{j=0}^{n-2} (P_{j-1}(\tau_i) - P_{j+1}(\tau_i)) P_j(t) + \sum_{j=n-1}^n P_{j-1}(\tau_i) P_j(t) \right).$$

□

Finally, it remains to define the coefficients

$$g_{i,j} = \frac{1}{2} (P_{j-1}(\tau_i) - P_{j+1}(\tau_i)), \quad j = 0, \dots, n-2, \quad g_{i,n-1} = \frac{1}{2} P_{n-2}(\tau_i), \quad g_{i,n} = \frac{1}{2} P_{n-1}(\tau_i)$$

for $i = 0, \dots, n-1$ and

$$g_{-1,j} = \frac{1}{2} \delta_{0,j}, \quad g_{n,j} = -\frac{1}{2} \delta_{0,j}, \quad j = 0, \dots, n,$$

so that the curve p can then be expressed in the Legendre basis as

$$p(t) = \sum_{j=0}^n c_j P_j(t), \quad c_j = \sum_{i=0}^n (g_{i-1,j} - g_{i,j}) p_i. \quad (9)$$

3. Linear-time evaluation

Given a GL curve p with control points $\mathbf{p} = (p_0, \dots, p_n)^T$, the alternative representation in (9) is nothing other than a change from the GL basis $\mathbf{F}^n = (F_0^n, \dots, F_n^n)^T$ to the Legendre basis $\mathbf{P}^n = (P_0, \dots, P_n)^T$, and the transformation matrix of this basis change is the matrix $M_n \in \mathbb{R}^{(n+1) \times (n+1)}$ with entries $m_{j,i} = g_{i-1,j} - g_{i,j}$ for $j, i = 0, \dots, n$. That is, instead of writing the curve as $p = \mathbf{p}^T \mathbf{F}^n$, we express it instead as $p = \mathbf{c}^T \mathbf{P}^n$, where $\mathbf{c} = (c_0, \dots, c_n)^T$ are the curve's Legendre coefficients, which can be written as $\mathbf{c} = M_n \mathbf{p}$.

Clearly, this conversion requires $O(n^2)$ operations, but it needs to be computed only once, whenever the number of control points changes. Afterwards, if the user interactively moves some control point p_i by some displacement vector Δ to $p_i + \Delta$, we keep $\mathbf{c}$ up-to-date by simply adding $\mathbf{m}_i \Delta^T$ to $\mathbf{c}$, where $\mathbf{m}_i$ denotes the i -th column of M_n , which can be done in $O(n)$ time.

The major advantage of the Legendre representation in (9) is that it can be used to compute $p(t)$ in $O(n)$ time for any $t \in [-1, 1]$. To this end, recall that the Legendre polynomials obey the three-term recurrence relation (Abramowitz and Stegun, 1965, Eq. (8.5.3)),

$$P_0(t) = 1, \quad P_1(t) = t, \quad P_i(t) = \alpha_i t P_{i-1}(t) - \beta_i P_{i-2}(t), \quad i \geq 2,$$

where the constants $\alpha_i = \frac{2i-1}{i}$ and $\beta_i = \frac{i-1}{i}$ for $i \geq 2$ can be precomputed and hard-coded or read from a file. We can then evaluate p with Algorithm 1 in linear time. For curves in $\mathbb{R}^2$, this algorithm requires $8n-4$ floating point operations. Hence, it performs almost as well as the most efficient polynomial evaluation algorithms, for example, evaluation in barycentric form, which requires $7n+9$ operations (Fuda et al., 2024).

Algorithm 1 GLEVAL($c_0, \dots, c_n, t$).

```

 $r_2 \leftarrow 1$ 
 $r_1 \leftarrow t$ 
 $p \leftarrow c_0 + r_1 \cdot c_1$ 
for  $j = 2, \dots, n$  do
     $r_j \leftarrow \alpha_j \cdot t \cdot r_1 - \beta_j \cdot r_2$ 
     $p \leftarrow p + r_j \cdot c_j$ 
     $r_2 \leftarrow r_1$ 
     $r_1 \leftarrow r_j$ 
return  $p$ 

```

4. GL- k curves

Modelling with GL curves is intuitive, but the curves have the tendency to “overshoot” the control polygon by quite some margin (see Fig. 2, right), which is caused by the fact that the GL basis functions F^n have rather big negative values (see Fig. 4, left). The motivation for our work lies in alleviating this effect.

For closed curves over $[0, 2\pi]$, Ramanantoanina and Hormann (2024) explore a trigonometric analogue of GL curves and modify them as follows. While trigonometric GL curves have tangent vectors at the equidistant nodes $\psi_i = \frac{2i+1}{n+1}\pi$ that are parallel to the edge vectors $v_i = p_{i+1} - p_i$, the modified curves have tangent vectors at the dual nodes $\phi_i = \frac{2i}{n+1}\pi$ that are parallel to the averages of two successive edge vectors, $v_i^1 = (v_{i-1} + v_i)/2 = (p_i + p_{i+1})/2 - (p_{i-1} + p_i)/2$. Our goal now is to develop and extend a similar modification for polynomial GL curves, which gives rise to the family of GL- k curves, where $k \in \mathbb{N}_0$.

Given the $n+1$ control points $p_0, \dots, p_n \in \mathbb{R}^2$, we recursively define the points

$$p_0^k = p_0^{k-1}, \quad p_i^k = \frac{p_{i-1}^{k-1} + p_i^{k-1}}{2}, \quad i = 1, \dots, n+k-1, \quad p_{n+k}^k = p_{n+k-1}^{k-1},$$

for $k = 1, 2, \dots$, where $p_i^0 = p_i$ for $i = 0, \dots, n$. Clearly, this averaging process can be written more compactly as $p^k = A_k p^{k-1}$, where

$$p^k = (p_0^k, \dots, p_{n+k}^k)^T, \quad A_k = \frac{1}{2} \begin{pmatrix} 2 & 0 & \dots & 0 \\ 1 & 1 & \dots & 0 \\ & \ddots & \ddots & \\ 0 & \dots & 1 & 1 \\ 0 & \dots & 0 & 2 \end{pmatrix} \in \mathbb{R}^{(n+k+1) \times (n+k)} \quad (10)$$

and $p^0 = p$. Now let p^k be the GL curve defined by $p^k = (p_0^k, \dots, p_{n+k}^k)^T$, that is, $p^k = (p^k)^T F^{n+k}$. It is clear that the tangents of p^k at the roots of P_{n+k} are parallel to the edge vectors $v_i^k = p_{i+1}^k - p_i^k$ for $i = 0, \dots, n+k-1$, and we observe that the overshooting effect diminishes as k grows (see Fig. 3, second row). This is essentially what we were aiming for, but p^k is a curve of degree $n+k$, even though it depends only on the $n+1$ control points p . The core idea of GL- k curves now is to approximate the GL curve p^k with a polynomial curve of degree n that is as close as possible to p^k and interpolates the same endpoints (see Fig. 3, third row).

Definition 1. Let $\mathcal{R}_k : \Pi_{n+k} \rightarrow \Pi_n$ be the *reduction operator* that projects any polynomial f of degree $n+k$ to the polynomial g of degree n that is closest to f in the L^2 -norm, subject to the endpoint constraints $g(-1) = f(-1)$ and $g(1) = f(1)$. The *GL- k curve* c^k is then defined as $c^k = \mathcal{R}_k[p^k]$.

The next lemma reveals that the operator $\mathcal{R}_k$ has a rather simple representation matrix with respect to the Legendre basis, which can be combined nicely with the Legendre representation of GL curves in (9).

Lemma 1. For any $k \in \mathbb{N}$, let $m = n+k$ and let $f \in \Pi_m$ by a degree- m polynomial with Legendre coefficients $f = (f_0, \dots, f_m)^T$. Then the Legendre coefficients $g = (g_0, \dots, g_n)^T$ of the degree- n polynomial $g = \mathcal{R}_k[f] \in \Pi_n$ are

$$g_i = f_i + \sum_{j=1}^k s_{ij} f_{n+j}, \quad i = 0, \dots, n,$$

where

$$s_{ij} = \frac{4i+2}{n+1} \cdot \begin{cases} 0, & \text{if } n+i+j \text{ is odd,} \\ \frac{1}{n}, & \text{if } n+i+j \text{ is even and } j \text{ is odd,} \\ \frac{1}{n+2}, & \text{if } n+i+j \text{ is even and } j \text{ is even.} \end{cases} \quad (11)$$

Hence, the representation matrix of $\mathcal{R}_k$ with respect to the Legendre basis is

$$R_k = [I_{n+1} | S] \in \mathbb{R}^{(n+1) \times (n+1+k)}, \quad (12)$$

where I_{n+1} is the identity matrix of size $n+1$ and the coefficients of $S \in \mathbb{R}^{(n+1) \times k}$ are the $s_{i,j}$ in (11) for $i = 0, \dots, n$ and $j = 1, \dots, k$.

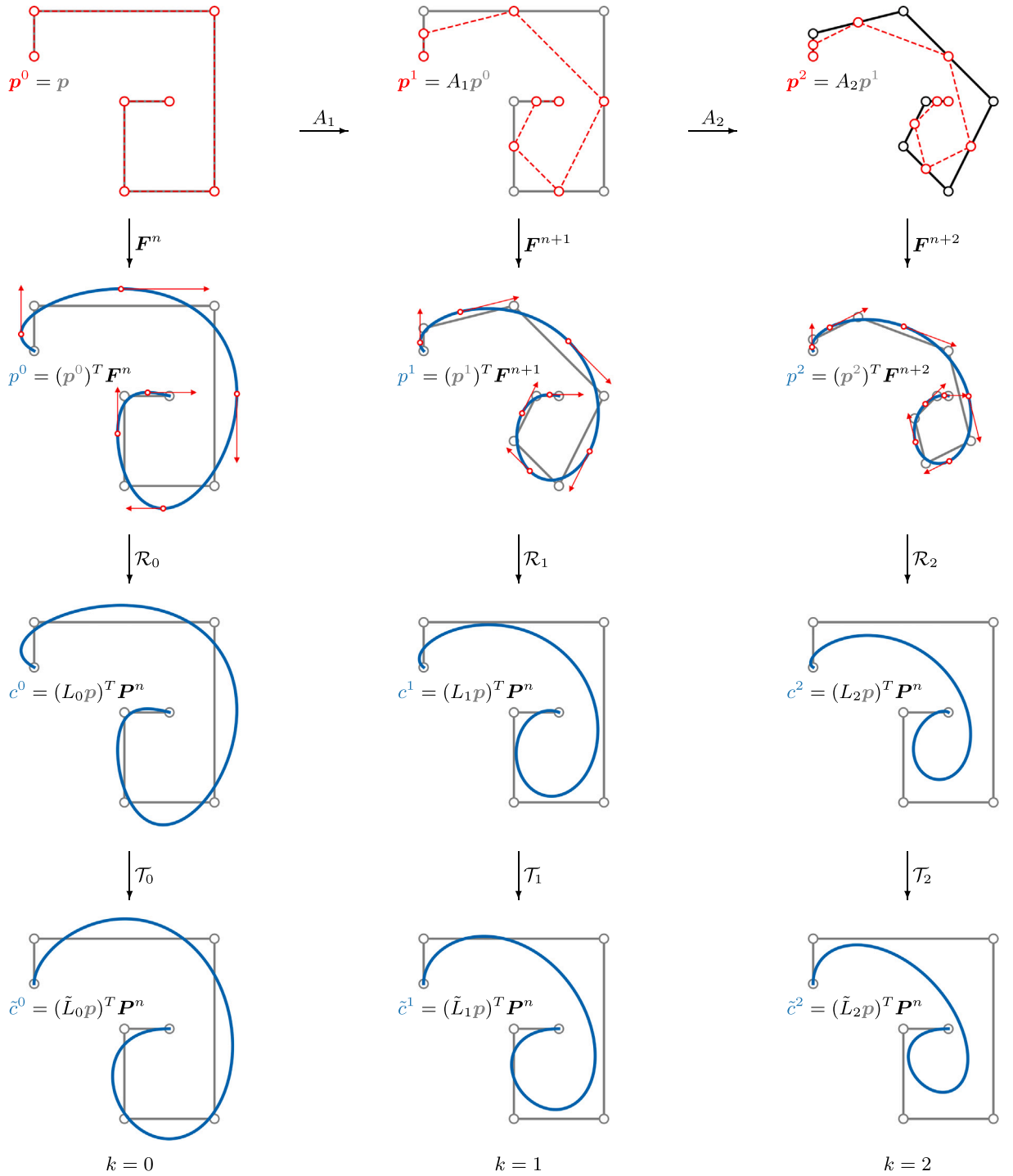


Fig. 3. Overview of the proposed construction of GL- k curves. Given a control polygon p , we smooth it k times (top, from left to right) by averaging neighbouring points. The GL curve p^k of degree $n+k$, defined by the resulting control polygon p^k (second row), is then projected to the space of degree- n polynomials with the same endpoints, using the reduction operator $\mathcal{R}_k$, to give the GL- k curve c^k (third row). Finally, the modified GL- k curve $\hat{c}^k$ (last row) can be generated by using the tangent operator $\mathcal{T}_k$ which finds the degree- n curve with the endpoint tangent property that is closest to c^k .

Proof. Let $\tilde{f} = (f_0, \dots, f_n)^T$ be the first $n+1$ Legendre coefficients of f . It is then well-known that $\tilde{f} \in \Pi_n$ with $\tilde{f}(t) = \sum_{i=0}^n f_i P_i(t)$ is the best L^2 -approximation of f in Π_n (Ascher and Greif, 2011). Therefore, instead of minimizing $\|g - f\|_2$ to find g as the closest degree- n polynomial to f in the L^2 -norm, we can determine g by minimizing $\|g - \tilde{f}\|_2$, subject to the endpoint constraints. By the

linearity of the L^2 -inner product,

$$\|g - \tilde{f}\|_2^2 = \langle g - \tilde{f}, g - \tilde{f} \rangle_2 = \sum_{i=0}^n \sum_{j=0}^n (g_i - f_i)(g_j - f_j) \langle P_i, P_j \rangle_2,$$

and since the Legendre polynomials are orthogonal with respect to this inner product and satisfy (Arfken et al., 2012, Eq. (12.47))

$$\langle P_i, P_j \rangle_2 = \int_{-1}^1 P_i(s) P_j(s) ds = \frac{2}{2i+1} \delta_{i,j},$$

we have

$$\|g - \tilde{f}\|_2^2 = \sum_{i=0}^n (g_i - f_i)^2 \frac{2}{2i+1} = (g - \tilde{f})^T W (g - \tilde{f}),$$

where $W \in \mathbb{R}^{(n+1) \times (n+1)}$ is the diagonal matrix

$$W = \text{diag}(w_0, \dots, w_n), \quad w_i = \frac{2}{2i+1}. \quad (13)$$

Since $P_i(-1) = (-1)^i$ and $P_i(1) = 1$ for any $i \geq 0$, we can write the endpoint constraints compactly as $C_{n+1}g = C_{m+1}f$, where

$$C_k = \begin{pmatrix} 1 & -1 & \dots & (-1)^{k-1} \\ 1 & 1 & \dots & 1 \end{pmatrix} \in \mathbb{R}^{2 \times k}, \quad k \in \mathbb{N}.$$

The Legendre coefficients g of the constrained L^2 -minimizer $g = \mathcal{R}_k[f]$ are then given as the stationary point of the Lagrangian function

$$\mathcal{L}(g, \lambda) = (g - \tilde{f})^T W (g - \tilde{f}) + \lambda^T (C_{n+1}g - C_{m+1}f)$$

with Lagrangian multiplier $\lambda \in \mathbb{R}^2$. Setting the partial derivatives of $\mathcal{L}$ with respect to g and λ to zero, we get the two equations

$$2W(g - \tilde{f}) + C_{n+1}^T \lambda = 0, \quad C_{n+1}g - C_{m+1}f = 0$$

and solving them in g and λ , we find that

$$g = \tilde{f} + W^{-1} C_{n+1}^T (C_{n+1} W^{-1} C_{n+1}^T)^{-1} (C_{m+1}f - C_{n+1}\tilde{f}).$$

Since $\tilde{f} = [I_{n+1} | \mathbf{0}_k] f$, where $\mathbf{0}_k$ is the zero matrix of size $(n+1) \times k$, and $C_{m+1} = [C_{n+1} | C'_k]$, where $C'_k = \text{diag}((-1)^{n+1}, 1) \cdot C_k$, we can write g as

$$g = [I_{n+1} | \mathbf{0}_k] f + V ([C_{n+1} | C'_k] - C_{n+1} [I_{n+1} | \mathbf{0}_k]) f = [I_{n+1} | W^{-1}V] f,$$

where $V = C_{n+1}^T (C_{n+1} W^{-1} C_{n+1}^T)^{-1} C'_k \in \mathbb{R}^{(n+1) \times k}$. To find the coefficients of V , note that

$$W^{-1} = \text{diag}\left(\frac{1}{w_0}, \dots, \frac{1}{w_n}\right),$$

hence

$$C_{n+1} W^{-1} C_{n+1}^T = \begin{pmatrix} \sum_{i=0}^n \frac{2i+1}{2} (-1)^{2i} & \sum_{i=0}^n \frac{2i+1}{2} (-1)^i \\ \sum_{i=0}^n \frac{2i+1}{2} (-1)^i & \sum_{i=0}^n \frac{2i+1}{2} \end{pmatrix} = \frac{n+1}{2} \begin{pmatrix} n+1 & (-1)^n \\ (-1)^n & n+1 \end{pmatrix}$$

and

$$(C_{n+1} W^{-1} C_{n+1}^T)^{-1} = \frac{2}{n(n+1)(n+2)} \begin{pmatrix} n+1 & (-1)^{n+1} \\ (-1)^{n+1} & n+1 \end{pmatrix}.$$

Multiplying from the left with C_{n+1}^T and from the right with C'_k gives

$$V = \frac{4}{n(n+1)(n+2)} \begin{pmatrix} v_0 \\ \vdots \\ v_n \end{pmatrix}$$

with row vectors

$$v_i = \begin{cases} (n+2, 0, n+2, 0, \dots), & \text{if } n+i \text{ is odd,} \\ (0, n, 0, n, \dots), & \text{if } n+i \text{ is even,} \end{cases}$$

which means that the coefficients $v_{i,j}$ of V for $i = 0, \dots, n$ and $j = 1, \dots, k$ are

$$v_{i,j} = \frac{4}{n+1} \begin{cases} 0, & \text{if } n+i+j \text{ is odd,} \\ \frac{1}{n}, & \text{if } n+i+j \text{ is even and } j \text{ is odd,} \\ \frac{1}{n+2}, & \text{if } n+i+j \text{ is even and } j \text{ is even.} \end{cases}$$

The statement then follows by noting the $s_{i,j}$ in (11) are the coefficients of $S = W^{-1}V$. □

Algorithm 2 LEGENDREREDUCE(k, f).

```

 $m \leftarrow \text{length}(f) - 1$ 
 $n \leftarrow m - k$ 
 $\sigma_e, \sigma_o \leftarrow 0$ 
for  $j = 1, \dots, k$  do
  if  $n + j$  is even then
     $\sigma_e \leftarrow \sigma_e + f_{n+j}$ 
  else
     $\sigma_o \leftarrow \sigma_o + f_{n+j}$ 
if  $n$  is even then
   $\sigma_e \leftarrow \sigma_e / (n + 1) / (n + 2)$ 
   $\sigma_o \leftarrow \sigma_o / n / (n + 1)$ 
else
   $\sigma_e \leftarrow \sigma_e / n / (n + 1)$ 
   $\sigma_o \leftarrow \sigma_o / (n + 1) / (n + 2)$ 
for  $i = 0, \dots, n$  do
  if  $i$  is even then
     $g_i \leftarrow f_i + (4i + 2)\sigma_e$ 
  else
     $g_i \leftarrow f_i + (4i + 2)\sigma_o$ 
return  $g$ 

```

Note that the reduction operator $\mathcal{R}_k$ with respect to the Legendre basis can be implemented with $O(n + k)$ instructions, as shown in Algorithm 2.

Let us summarize the construction of GL- k curves (see Fig. 3). Given the $n + 1$ control points $\mathbf{p} = (p_0, \dots, p_n)^T$, we average these points k times to get the points $\mathbf{p}^k = A_k \cdots A_1 \mathbf{p}$. We then consider the GL curve $\mathbf{p}^{n+k}$ for these points, with coefficients $M_{n+k} \mathbf{p}^k$ in the Legendre basis. We finally degree-reduce the latter with the reduction operator $\mathcal{R}_k$ to get the Legendre coefficients $\mathbf{c}^k = L_k \mathbf{p}$ of the GL- k curve c^k , where the matrix $L_k = R_k M_{n+k} A_k \cdots A_1 \in \mathbb{R}^{(n+1) \times (n+1)}$ can be precomputed. As for GL curves, the initial conversion from $\mathbf{p}$ to $\mathbf{c}^k$ requires $O(n^2)$ operations. Any subsequent change of some control point p_i leads to an $O(n)$ update of $\mathbf{c}^k$, and the actual evaluation of c^k happens in linear time with Algorithm 1. Note that the GL-0 curve c^0 is identical to the GL curve $\mathbf{p}^n$, because $\mathcal{R}_0$ is just the identity.

5. Properties of GL- k curves

So far, we have considered GL- k curves in terms of the Legendre coefficients $\mathbf{c}^k$ with respect to the Legendre basis,

$$c^k(t) = (\mathbf{c}^k)^T \mathbf{P}^n = \sum_{i=0}^n c_i^k P_i(t),$$

but since $\mathbf{c}^k = L_k \mathbf{p}$, we can also express them in terms of the control points $\mathbf{p}$ with respect to the functions $\mathbf{C}^k = (C_0^k, \dots, C_n^k)^T = (L_k)^T \mathbf{P}^n$ as

$$c^k(t) = \mathbf{p}^T \mathbf{C}^k = \sum_{i=0}^n p_i C_i^k(t).$$

We shall now show two important properties of the functions C_i^k .

Proposition 3. *The functions $C_0^k, \dots, C_n^k$ are polynomials and form a basis of Π_n .*

Proof. It is clear that the functions $C_0^k, \dots, C_n^k$ are polynomials of degree n , because they are linear combinations of the Legendre polynomials $P_0, \dots, P_n$. It remains to show that $L_k = R_k M_{n+k} A_k \cdots A_1$ has full rank and thus is a change-of-basis matrix. However, this follows from the facts that the averaging matrices A_k in (10) have full column rank, the reduction matrix R_k in (12) has full row rank, and the matrix M_{n+k} which describes the basis change from the GL basis to the Legendre basis has full rank (Moon et al., 2023). $\square$

Proposition 4. *The polynomials $C_0^k, \dots, C_n^k$ form a partition of unity.*

Proof. Using the notation of Proposition 3, we need to show that $\mathbf{1}_{n+1}^T \mathbf{C}^k = (\mathbf{C}^k)^T \mathbf{1}_{n+1} = 1$, where $\mathbf{1}_{n+1} = (1, \dots, 1)^T \in \mathbb{R}^{n+1}$. Using the change-of-basis matrix L_k and observing that the entries in each row of A_i sum to one, that is, $A_i \mathbf{1}_{n+i} = \mathbf{1}_{n+i+1}$, we have

$$(\mathbf{C}^k)^T \mathbf{1}_{n+1} = (\mathbf{P}^n)^T R_k M_{n+k} A_k \cdots A_1 \mathbf{1}_{n+1} = (\mathbf{P}^n)^T R_k M_{n+k} \mathbf{1}_{n+k+1}.$$

Now, recall that M_{n+k} is the change-of-basis matrix from the GL basis $\mathbf{F}^{n+k}$ to the Legendre basis $\mathbf{P}^{n+k}$ and that the polynomials $\mathbf{F}^{n+k}$ form a partition of unity (Moon et al., 2023). Therefore, $M_{n+k} \mathbf{1}_{n+k+1}$ is the vector of Legendre coefficients of the constant 1 function,

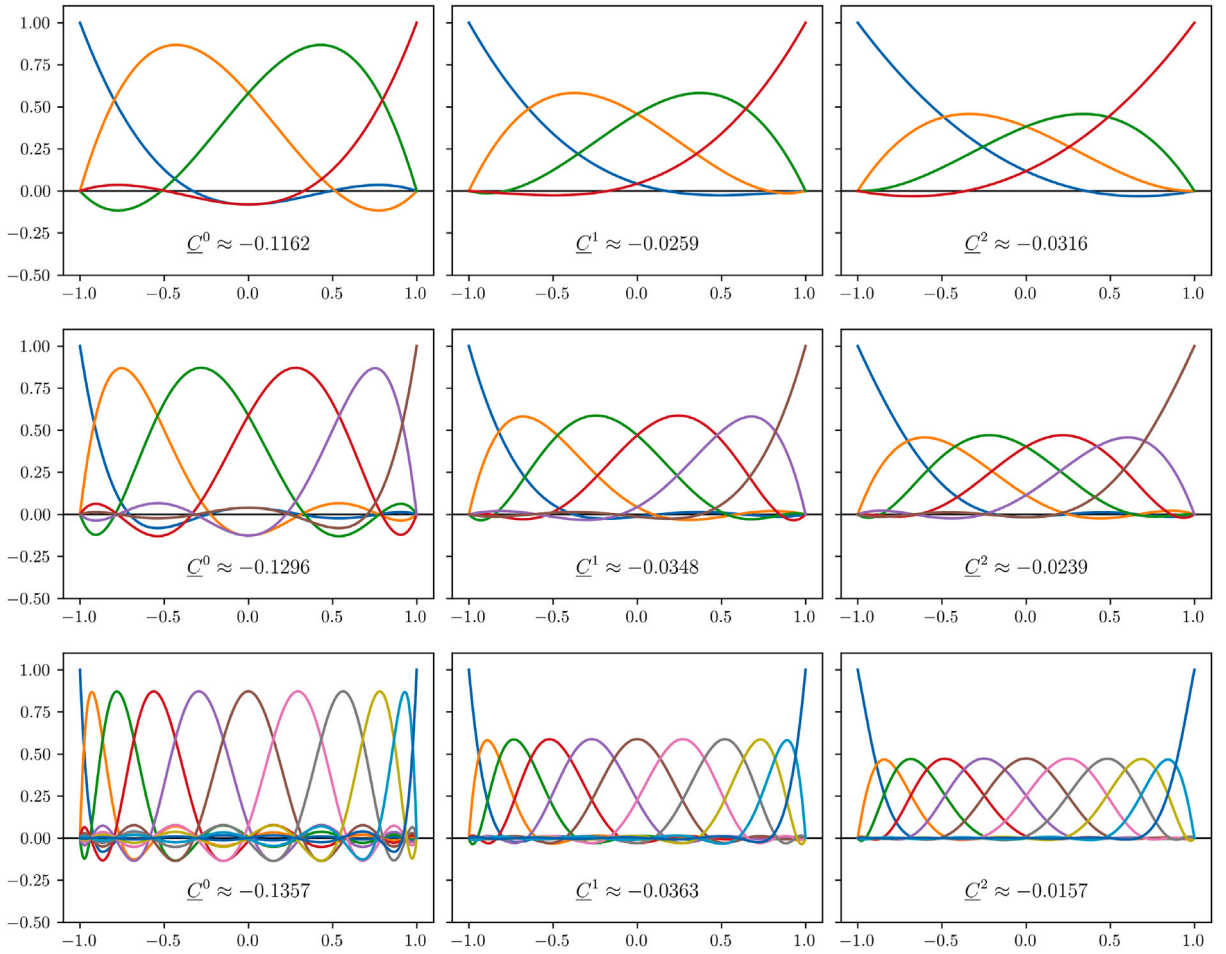


Fig. 4. Basis functions $C_0^k, \dots, C_n^k$ of GL- k curves for degrees $n = 3, 5, 10$ (top to bottom) and $k = 0, 1, 2$ (left to right).

but as this function is just the Legendre polynomial P_0 , this coefficient vector is just $e_0^{n+k} = (1, 0, \dots, 0)^T \in \mathbb{R}^{n+k+1}$. By the structure of R_k in (12), we then have

$$(C^k)^T \mathbf{1}_{n+1} = (P^n)^T R_k e_0^{n+k} = (P^n)^T e_0^n = P_0,$$

which proves the claim. $\square$

It follows from these properties that all polynomial curves of degree n can be described as GL- k curves and that every GL- k curve is invariant under affine transformations of its control polygon, which are two key properties that GL- k curves share with Bézier curves. Unlike Bézier curves, GL- k curves do not have the *convex hull property*, because the basis functions C_i^k are *not* non-negative (see Fig. 4). However, we observe (and further experiments reveal) that the lower bound $\underline{C}^k = \min\{C_i^k(t) : i = 0, \dots, n, t \in [-1, 1]\} < 0$ seems to converge rather quickly to 0 as k increases (at least if $n \geq 2k$). In other words, the basis functions C_i^k are obtained by forming convex combinations of the GL basis of degree $n+k$ and then projecting them back to degree n . The convex combinations increase the lower bound, while the back projection partially counteracts this effect. In practice, for small values of k , the influence of the back projection remains rather limited. Furthermore, unlike GL curves, the existence of a derivative vector parallel to each edge of the control polygon at very specific nodes is no longer guaranteed for GL- k curves with $k > 0$.

6. Endpoint tangents

Another common property, which is key for intuitive design, is the endpoint interpolation property. In contrast to Bézier curves, GL- k curves do not have the endpoint tangent property, that is, the vectors $(c^k)'(-1)$ and $(c^k)'(1)$ are not necessarily parallel to the vectors $p_1 - p_0$ and $p_n - p_{n-1}$, respectively.

To restore this property, we can follow the steps in the proof of Lemma 1 to define the *tangent operator* $\mathcal{T}_k$ that projects the GL- k curve c^k to the *modified* GL- k curve $\tilde{c}^k = \mathcal{T}_k[c^k] = \tilde{c}^k P^n$ which minimizes $\|\tilde{c}^k - c^k\|_2$, subject to the endpoint interpolation and the

endpoint tangent property. Since $P'_i(-1) = (-1)^{i+1}d_i$ and $P'_i(1) = d_i$, where $d_i = i(i+1)/2$ (Olver et al. (2010), Eq. (18.9.15); Szegő (1975), Eq. (4.1.1)) and recalling that $c^k = L_k p$, we can write the constraints compactly as $D\tilde{c}^k = E p$, where

$$D = \begin{pmatrix} 1 & -1 & \cdots & (-1)^n \\ 1 & 1 & \cdots & 1 \\ -d_0 & d_1 & \cdots & (-1)^{n+1}d_n \\ d_0 & d_1 & \cdots & d_n \end{pmatrix}, \quad E = \begin{pmatrix} \begin{pmatrix} 1 & -1 & \cdots & (-1)^n \\ 1 & 1 & \cdots & 1 \end{pmatrix} L_k \\ \begin{pmatrix} -\eta_1 & \eta_1 & 0 & \cdots & 0 & 0 \\ 0 & 0 & 0 & \cdots & -\eta_2 & \eta_2 \end{pmatrix} \end{pmatrix}$$

with $\eta_1, \eta_2 > 0$ denoting the scaling factors between the endpoint tangents of $\tilde{c}^k$ and the vectors $p_1 - p_0$ and $p_n - p_{n-1}$. As in the proof of Lemma 1, one can show that the solution of this constrained optimization problem is

$$\tilde{c}^k = \tilde{L}_k p, \quad \tilde{L}_k = L_k + W^{-1} D^T (D W^{-1} D^T)^{-1} (E - D L_k),$$

with W as in (13). The corresponding curve $\tilde{c}^k$ depends on the scaling factors η_1 and η_2 , and the actual choice can be left to the user. Based on our experiments, a reasonable default value is to set $\eta_1 = \eta_2 = 1/\omega_0$ where ω_0 is the first Legendre weight $\omega_0 = \int_{-1}^1 \ell_0(s) ds$, so that $(\tilde{c}^k)'(-1) = p'(\tau_0)$ and $(\tilde{c}^k)'(1) = p'(\tau_{n-1})$ (cf. (3)). We used this default value in Fig. 3 (last row) and Fig. 8.

7. Results

We implemented GL- k (and modified GL- k) curves in C++ on an Ubuntu laptop with an 8-core Intel i7-10510U CPU at 1.80 GHz and 16 GB of RAM. The code was compiled with CMake using the `-O2` optimization flag. As described above, each time the number of control points changes, we compute the $(n+1) \times (n+1)$ matrix L_k (or $\tilde{L}_k$) and the Legendre coefficients c^k (or $\tilde{c}^k$) in a preprocessing step which is quadratic in n . During a modelling session, these coefficients are updated in linear time whenever the user moves a control point. For plotting the curve, we evaluate it at 1000 equidistant parameter values using Algorithm 1, which also has linear runtime. For comparison, cubic B-spline curves are evaluated using the *Eigen* module (Guennebaud et al., 2021).

Fig. 5 shows a comparison of GL- k and cubic B-spline curves. We observe that GL-1 curves typically behave and appear like cubic B-spline curves with uniformly distributed knots. As k increases, GL- k curves keep “shrinking” away from the control polygon (like Bézier curves, but much less so) and behave like higher-degree B-spline curves. In particular, they fail to capture the intended shape if the control polygon contains zig-zag sections (Fig. 5, right). This is less of an issue with GL-0 curves, the classical GL curves. On the other hand, manipulating GL-0 curves can cause unintended artefacts as the basis functions are less local than the basis functions of GL- k curves for $k > 0$ (see Fig. 4).

We further investigate the similarity between modified GL-1 and cubic B-spline curves by looking at their curvature profiles. As can be seen in Fig. 6, the curvature of the GL-1 curve in this example is very similar to that of the cubic B-spline curve with the same control polygon, but with two advantages. On the one hand, it oscillates less and has fewer inflection points. On the other hand, it is continuously differentiable, while the curvature of the cubic B-spline curve is only C^0 at the knots.

Fig. 7 reports the runtimes of our implementation. For this experiment, we generated 1000 random curves with $n+1$ control points, for $n = 3, \dots, 19$, and evaluated them at 1000 equidistant parameter values. The plots confirm our expectations. On the one

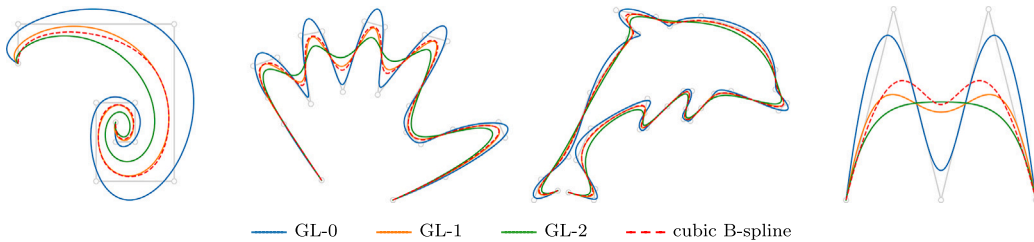


Fig. 5. Comparison of GL- k curves for $k = 0, 1, 2$ and cubic B-spline curves with respect to uniform nodes.

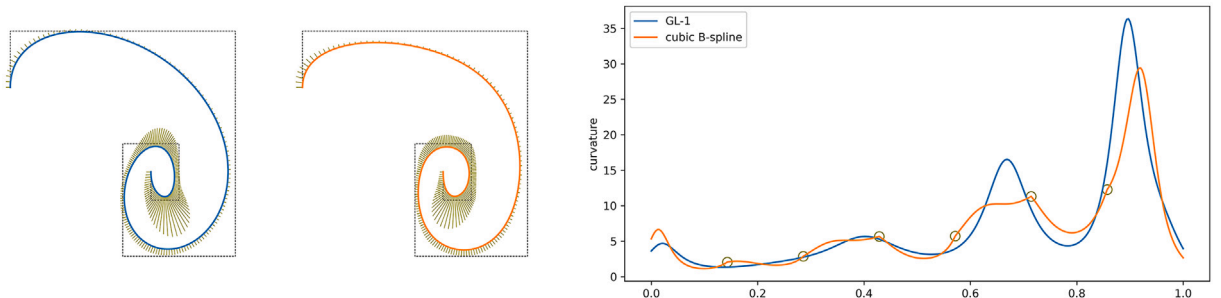


Fig. 6. Curvature comb plots of a modified GL-1 (left) and a cubic B-spline curve (centre) with the same control polygon along with plots of both curvature functions (right). The circles indicate where the curvature of the cubic B-spline curve is only C^0 .

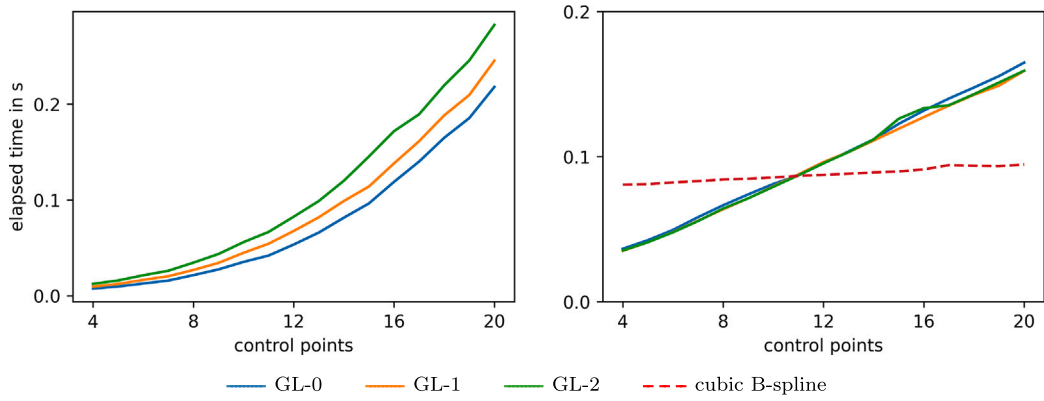


Fig. 7. Runtime comparison for evaluating GL-0, GL-1, GL-2, and cubic B-spline curves. While the preprocessing step that converts control points to Legendre coefficients is quadratic in the number of control points (left), the actual evaluation of GL- k curves has linear runtime and is faster than the evaluation of cubic B-spline curves for up to 10 control points (right).

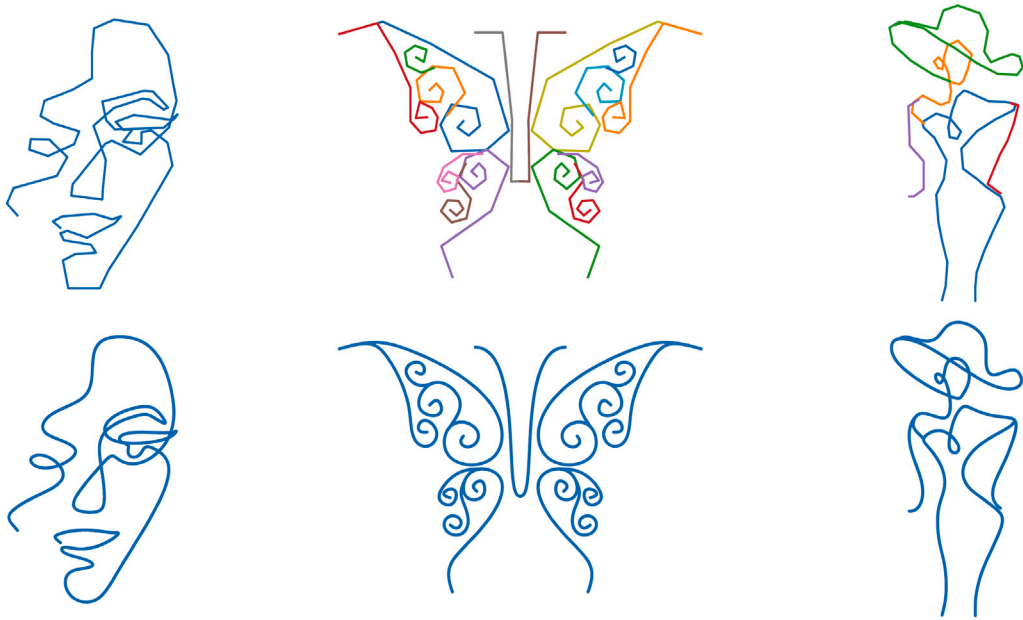


Fig. 8. Examples of line art with modified GL-1 curves. The face stroke is generated using a single curve of degree 75. The butterfly and the lady are examples where several GL-1 curves were used to design a more complicated artwork.

hand, the initial conversion from control points to Legendre coefficients is quadratic in n and grows with k . On the other hand, the evaluation of GL- k curves is linear in n and *independent of k* , while cubic B-splines can be evaluated essentially in constant time. We observe that evaluating GL- k curves in the Legendre basis with [Algorithm 1](#) is more efficient than evaluating cubic B-spline curves for up to 10 control points, and even for 20 control points, it is less than twice as expensive.

8. Conclusion

GL- k curves constitute a novel family of polynomial curves that offer an intuitive tool for curve modelling. Unlike Bézier curves, the corresponding basis functions are not non-negative (see [Fig. 4](#)), and consequently a GL- k curve is not guaranteed to remain within the convex hull of its control points. This is, however, a trade-off that allows the curve to follow the shape of its control polygon more closely, especially for high-degree curves.

Based on our experiments, our favourite curves are modified GL-1 curves, which offer the best design experience and can be used to quickly design single-stroke polynomial drawings (see [Fig. 8](#), left). Their intuitive manipulation is particularly useful for manual refinement. Instead of relying on a single high-degree curve, one may also generate more intricate artwork by using several curves (see [Fig. 8](#), middle and right). Thanks to the endpoint interpolation and the endpoint tangent property, it is easy to ensure C^0 or C^1 continuity at corners and branching points.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgement

We thank the anonymous reviewers for their valuable comments and suggestions, which have helped improve this paper. This work was supported by the [Swiss National Science Foundation](#) (SNF) under project No. [188577](#).

Data availability

No data was used for the research described in the article.

References

- Abramowitz, M., Stegun, I.A. (Eds.), 1965. *Handbook of Mathematical Functions with Formulas, Graphs, and Mathematical Tables*, vol. 55 Applied Mathematics Series. Dover Publications, New York.
- Arfken, G.B., Weber, H.J., Harris, F.E., 2012. *Mathematical Methods for Physicists: A Comprehensive Guide*, seventh ed. Academic Press, Oxford.
- Ascher, U.M., Greif, C., 2011. Best approximation. In: Chapter 12: A First Course in Numerical Methods. Society for Industrial and Applied Mathematics, Philadelphia. Computational Science & Engineering, pp. 365–382. <https://doi.org/10.1137/9780898719987.ch12>
- Fuda, C., Ramanantoanina, A., Hormann, K., 2024. A comprehensive comparison of algorithms for evaluating rational Bézier curves. *Dolomites Res. Notes Approx.* 17, 56–79. <https://doi.org/10.14658/PUPJ-DRNA-2024-3-9>
- Guennebaud, G., Jacob, B., et al., 2021. Eigen 3.4.0. <http://eigen.tuxfamily.org> (Online; accessed 11 April 2026).
- Hartley, P.J., Judd, C.J., 1978. Parametrization of Bézier-type B-spline curves and surfaces. *Comput.-aided Des.* 10, 130–134. [https://doi.org/10.1016/0010-4485\(78\)90090-8](https://doi.org/10.1016/0010-4485(78)90090-8)
- Henrici, P., 1964. *Elements of Numerical Analysis*. John Wiley and Sons, New York.
- Jiang, B., Chen, R., 2025. G^2 interpolating spline with local maximum curvature. *ACM Trans. Graph.* 44, 229, 13 pages. <https://doi.org/10.1145/3763316>
- Kim, S.H., Moon, H.P., 2017. Rectifying control polygon for planar Pythagorean hodograph curves. *Comput. Aided Geom. Des.* 54, 1–14. <https://doi.org/10.1016/j.cagd.2017.03.016>
- Kim, S.H., Moon, H.P., 2019. Gauss–Lobatto polygon of Pythagorean hodograph curves. *Comput. Aided Geom. Des.* 74, 101768, 20 pages. <https://doi.org/10.1016/j.cagd.2019.101768>
- Lee, E.T.Y., 1989. Choosing nodes in parametric curve interpolation. *Comput.-aided Des.* 21, 363–370. [https://doi.org/10.1016/0010-4485\(89\)90003-1](https://doi.org/10.1016/0010-4485(89)90003-1)
- Moon, H.P., Kim, S.H., Kwon, S.-H., 2020. Shape analysis of planar PH curves with the Gauss–Legendre control polygons. *Comput. Aided Geom. Des.* 82, 101915, 18 pages. <https://doi.org/10.1016/j.cagd.2020.101915>
- Moon, H.P., Kim, S.H., Kwon, S.-H., 2023. Gauss–Legendre polynomial basis for the shape control of polynomial curves. *Appl. Math. Comput.* 451, 127995, 16 pages. <https://doi.org/10.1016/j.amc.2023.127995>
- Olver, F.W.J., Lozier, D.W., Boisvert, R.F., Clark, C.W., 2010. *NIST Handbook of Mathematical Functions*. Cambridge University Press, New York.
- Prautzsch, H., Boehm, W., Paluszny, M., 2002. *Bézier and B-Spline Techniques*. Mathematics and Visualization, Springer, Berlin, Heidelberg. <https://doi.org/10.1007/978-3-662-04919-8>
- Ramanantoanina, A., Hormann, K., 2024. Trigonometric tangent interpolating curves. In: Chen, R., Ritschel, T., Whiting, E. (Eds.), *Pacific Graphics Conference Papers and Posters*, Huangshan, China, pp. 1–8. <https://doi.org/10.2312/pg.20241297>
- Runge, C., 1901. Über empirische Funktionen und die Interpolation zwischen äquidistanten Ordinaten. *Z. Math. Phys.* 46, 224–243.
- Simon, B., 2008. The Christoffel–Darboux kernel. In: *Perspectives in Partial Differential Equations, Harmonic Analysis and Applications*. American Mathematical Society, Providence, pp. 295–336. <https://doi.org/10.1090/pspum/079>
- Szegő, G., 1975. *Orthogonal Polynomials*, fourth ed. vol. 23 Colloquium Publications. American Mathematical Society, Providence.
- Yan, Z., Schiller, S., Wilensky, G., Carr, N., Schaefer, S., 2017. κ -curves: interpolation at local maximum curvature. *ACM Trans. Graph.* 36, 129, 7 pages. <https://doi.org/10.1145/3072959.3073692>